Synesthesia: Translating a Single Image into a Sung Song and Album Art via Three Cooperating Foundation Models

Dongjun Kim, 2026020940
Department of Computer Science and Engineering, Korea University
Deep Learning, Final Project

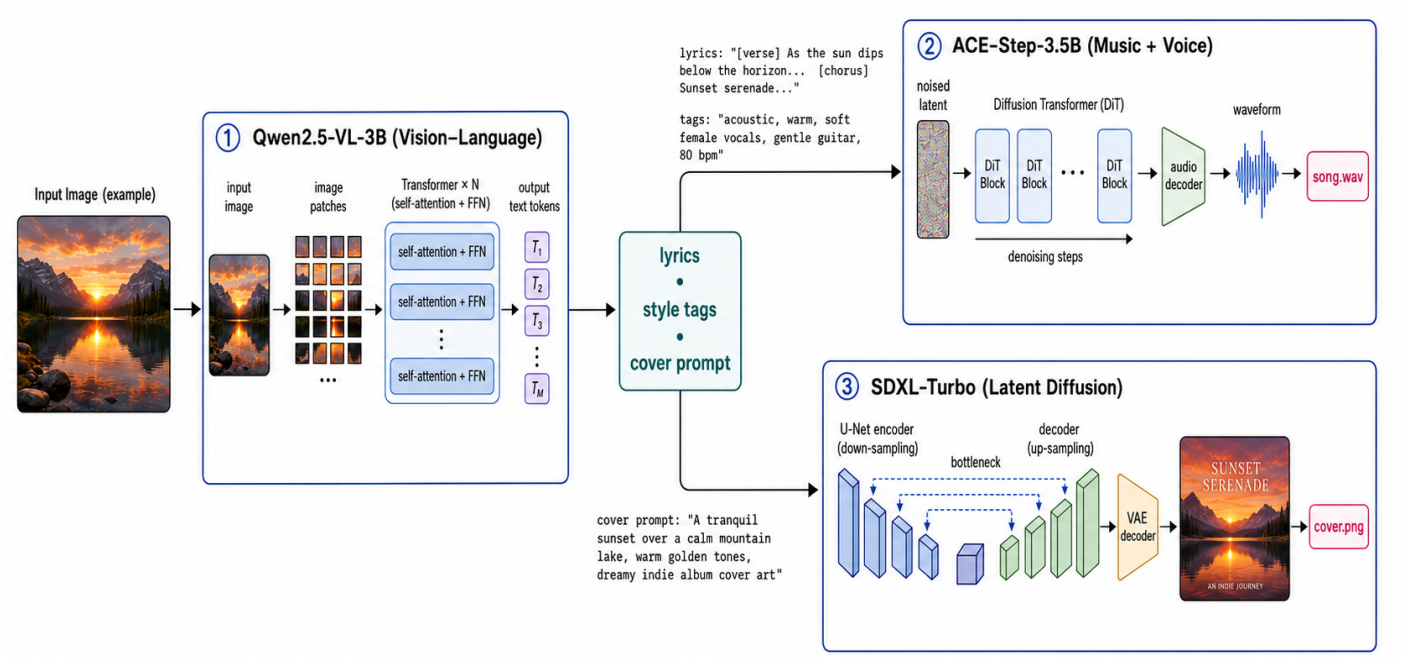

Fig. 1. The Synesthesia pipeline. A vision-language model (Qwen2.5-VL) writes lyrics and style tags from the input image; ACE-Step sings them into a full song; and SDXL-Turbo paints the album cover. All three models run locally from the HuggingFace Hub.

Abstract*—We present Synesthesia, an interactive system that transforms a single input image into an original, sung song together with a matching album-cover artwork. The system orchestrates three open foundation models, each operating in a distinct modality: a vision-language model (Qwen2.5-VL) interprets the image's mood and writes structured lyrics and style tags; a music-and-voice generation model (ACE-Step) renders those lyrics into a complete song with vocals; and a text-to-image diffusion model (SDXL-Turbo) paints an album cover. All model weights are obtained from the HuggingFace Hub and executed locally on a single 12 GB laptop GPU, with no reliance on external APIs. We describe a streaming web interface that surfaces each model's output as soon as it is produced, a memory-management strategy that fits three multi-billion-parameter models within 12 GB, and a process-isolation design that resolves conflicting library dependencies. We further report the engineering obstacles encountered and their solutions. The result is a reproducible, three-minute live demonstration of cooperative multimodal generation.*

Index Terms*—Foundation models, multimodal generation, vision-language models, music generation, text-to-image synthesis, system design.*

I. INTRODUCTION

Recent years have produced powerful foundation models in nearly every modality: vision-language models that describe and reason about images, generative models that compose music with singing voices, and diffusion models that synthesize photorealistic or stylized imagery. While each model is typically studied in isolation, an underexplored opportunity lies in composing several of them so that the output of one becomes the input of the next, yielding an end-to-end experience that no single model could deliver.

We explore this opportunity through a deliberately creative task: given a single photograph, produce an original song—with the lyrics actually sung—and an album cover inspired by the image. The name Synesthesia reflects the cross-modal nature of the system, which translates the visual content and mood of an image into sound and new imagery.

The system is designed under four practical constraints derived from the project requirements: it must use three or more foundation models; all models must run locally from the HuggingFace Hub without external APIs; the entire pipeline must fit on one 12 GB laptop GPU; and it must support a live, reproducible demonstration.

This paper makes the following contributions. (1) We design a cooperative three-model pipeline spanning vision-language understanding, music-and-voice generation, and image generation, in which each model occupies a distinct modality and role. (2) We build a streaming web interface that reveals each model's intermediate output in real time, making the cooperation visible to

the audience. (3) We describe engineering solutions for fitting the models within 12 GB and for isolating conflicting library dependencies. (4) We provide a fully reproducible open implementation.

II. SYSTEM ARCHITECTURE

The pipeline executes three stages in sequence, passing a structured intermediate representation between them.

A. Stage 1: Vision-Language Understanding

The first stage uses Qwen2.5-VL-3B-Instruct, a vision-language model, to both see and write. Given the input image, it produces a single structured object containing a song title, a set of mood words, a genre, a tempo, two verses and a chorus of lyrics annotated with section tags, a comma-separated list of style tags for the music model, and a prompt describing album-cover art. Using one vision-language model for both perception and lyric writing keeps the model count meaningful: a reviewer cannot object that a capable vision-language model was left unused, and the three models retain non-overlapping roles.

B. Stage 2: Song Synthesis

The second stage uses ACE-Step, a 3.5-billion-parameter music foundation model that generates a complete song—vocals and accompaniment together—from lyrics and style tags. Unlike a text-to-speech model, ACE-Step renders the lyrics as actual singing, which is essential to the perceived quality of the demonstration.

C. Stage 3: Album-Cover Generation

The third stage uses SDXL-Turbo, a distilled text-to-image diffusion model, to paint a 512x512 album cover from the cover prompt. Because image generation is a modality distinct from music, the third model adds visual impact without duplicating the role of the others.

III. IMPLEMENTATION

A. Streaming Interface

A FastAPI server drives the pipeline and streams newline-delimited JSON events to the browser, emitting one event as each model finishes. The single-page interface fills in three cards—one per model—so that the audience watches the lyrics appear, then hears the song, then sees the cover, making the cooperation of the models visible.

B. Memory Management

Although the three models together exceed 12 GB, the pipeline is strictly sequential. Each model is released from GPU memory once its stage completes, so peak usage stays within the budget of a single laptop GPU.

C. Process Isolation

ACE-Step pins an older release of the transformers library that is incompatible with the version required by Qwen2.5-VL. To resolve this, ACE-Step is installed in a separate conda environment and invoked as a subprocess; lyrics are passed through a temporary file to avoid shell-quoting issues. The main environment hosts the vision-language and image-generation models.

D. Robust Output Parsing

The vision-language model occasionally truncates its structured output or emits lyrics whose section markers contain bracket characters, both of which break strict JSON parsing. We raise the generation budget and add a regular-expression salvage parser that recovers each field even from malformed or truncated output.

IV. ENGINEERING CHALLENGES

Several practical obstacles arose during development. First, the library-version conflict between the singer and the vision-language model was resolved by environment isolation and subprocess invocation. Second, the audio backend in the installed version of the deep-learning framework routed file writes through a codec that lacks a Windows build; we patched the save routine to use a soundfile-based writer instead. Third, the singer's package on the public index was broken, so it was installed from source, with a locale workaround to avoid a text-decoding error during setup. Fourth, the parsing failures described above were addressed with a salvage parser. Finally, we verified that the laptop GPU, which idles at a low clock, boosts correctly under load, confirming that observed idle clocks were not a hardware limitation.

V. DEMONSTRATION AND RESULTS

The system produces a 25-second, 48 kHz stereo song and a 512x512 album cover for an arbitrary uploaded image. After weights are cached, the song model loads in roughly twelve to nineteen seconds and the full pipeline completes within the time budget of a short live demonstration. In a representative run on a sunset photograph, the vision-language model produced a song titled "Sunset Serenade" in an acoustic style with two verses and a chorus, which the singer rendered with soft vocals and gentle accompaniment while the diffusion model generated a matching cover.

A typical three-minute demonstration proceeds as follows: the presenter uploads an image; the lyrics and style appear as the vision-language model finishes; the sung song is played; and finally the album cover is revealed, closing with a one-sentence mapping back to the three foundation models.

VI. LIMITATIONS AND FUTURE WORK

Song quality depends on the style tags and the number of diffusion steps used by the singer, which trade quality against latency. The song length is configurable but longer songs increase generation time. Because the singer supports multiple languages, the system can be extended to generate lyrics and vocals in Korean, which we leave for future work.

VII. CONCLUSION

Synesthesia demonstrates that independently trained foundation models in different modalities can be composed into a single, coherent creative experience that runs locally on commodity hardware. By having a vision-language model see and write, a music model sing, and a diffusion model paint, the system turns one image into a complete song and its album art, while the streaming interface makes the cooperation of the models legible to an audience.

REFERENCES

[1] Qwen Team, "Qwen2.5-VL Technical Report," arXiv preprint, 2025.

[2] ACE-Step, "ACE-Step: A Step Towards Music Generation Foundation Models," 2025. [Online]. Available: https://github.com/ace-step/ACE-Step

[3] A. Sauer, D. Lorenz, A. Blattmann, and R. Rombach, "Adversarial Diffusion Distillation," arXiv preprint arXiv:2311.17042, 2023.

[4] J. Copet et al., "Simple and Controllable Music Generation," in Proc. Advances in Neural Information Processing Systems (NeurIPS), 2023.

[5] R. Rombach, A. Blattmann, D. Lorenz, P. Esser, and B. Ommer, "High-Resolution Image Synthesis with Latent Diffusion Models," in Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition (CVPR), 2022.

[6] T. Wolf et al., "Transformers: State-of-the-Art Natural Language Processing," in Proc. Conf. Empirical Methods in Natural Language Processing (EMNLP): System Demonstrations, 2020.